BUILD GUIDE · FOLLOW THIS ON YOUR OWN

Build the whole thing, end to end

From an empty machine to a running system: a typed FastAPI service, a PostgreSQL database, a Next.js page that talks to it, all started by one command and pushed to Git. Every file is here in full, with comments. Nothing is left as an exercise.

HOW TO USE THIS GUIDE

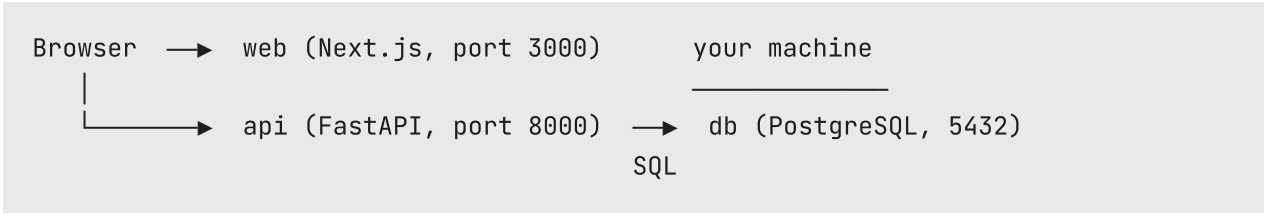
Work top to bottom. Each part ends with a **Checkpoint** — something you can see with your own eyes before moving on. If a checkpoint fails, stop there and fix it; a broken foundation gets much more expensive three parts later. Type the code rather than pasting it where you can, and read the comments as you go.

Total time from scratch: about two hours at a comfortable pace. You need an internet connection and roughly 3 GB of free disk space.

WHAT YOU ARE BUILDING

The finished system

A loan-application intake service. A person types a name into a web page and submits it; the page sends it to a Python API; the API stores it in PostgreSQL; the page reads the list back. Three programs, one command to start them all.



The browser reaches `web` and `api` on `localhost`. The api reaches the database by the name `db`, because they share a private Docker network. That distinction matters later — it is the single most common thing to get wrong.

`loan-intake/` every file you will create

```
loan-intake/
├── docker-compose.yml    # starts all three services together
├── README.md             # the one run command
├── .gitignore            # what must never be committed
├── api/
│   ├── Dockerfile        # how to build the api image
│   ├── requirements.txt  # pinned Python dependencies
│   ├── db.py             # database connection helper
│   ├── main.py           # the FastAPI service
│   └── test_main.py      # pytest tests
├── db/
│   └── init.sql          # schema + seed rows, run on first boot
└── web/
    ├── Dockerfile        # how to build the web image
    ├── package.json      # Node dependencies + scripts
    ├── tsconfig.json     # TypeScript settings
    ├── next.config.js    # Next.js settings
    └── app/
        ├── layout.tsx    # the page shell
        └── page.tsx      # the page itself
```

Sixteen files. You will write fifteen of them by hand and generate one.

PART 0

Prerequisites

Four tools. Install them in this order, and verify each one before installing the next — that way a failure is always attributable to the thing you just did.

1 Python 3.11 or later

Download from python.org/downloads and run the installer. **On Windows, tick "Add python.exe to PATH"** on the first screen — if you miss it, the `python` command will not be found afterwards and the simplest repair is to re-run the installer. On macOS you may prefer `brew install python@3.12`; on Ubuntu, `sudo apt install python3.12 python3.12-venv` (the `-venv` package is separate and you need it).

```
python --version
# Windows, if that fails:  py --version
# macOS/Linux, if that fails:  python3 --version
```

You want `Python 3.11.x` or higher. Note which command worked — `python`, `py` or `python3` — and use it wherever this guide says `python`.

2 Docker Desktop

Download from docker.com, install, then **launch the application** and leave it running. This last part is not optional: the Docker commands are a client that talks to a background engine, and the engine only runs while Docker Desktop is open. On Windows it will ask to enable WSL 2 — accept, and restart if prompted.

```
docker --version
docker compose version
```

Both must answer. Compose ships inside Docker Desktop, so there is nothing extra to install — but it is a separate plugin, so check it separately. Note the **space** in `docker compose`; older tutorials write `docker-compose` with a hyphen, which is the retired v1.

IF IT BREAKS

`Cannot connect to the Docker daemon` or `error during connect` means the engine is not running — open Docker Desktop and wait for its status to go green. On Linux there is no Desktop app: install the engine and the Compose plugin, then add yourself to the `docker` group (`sudo usermod -aG docker $USER`) and log out and back in, or every command will need `sudo`.

3 Git

From git-scm.com (macOS: also available via `xcode-select --install`). Then set your identity once — commits are refused without it.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4 An editor, and Node (optional)

Use VS Code with the official **Python** extension — it is what gives you the type-hint autocomplete and inline error squiggles this guide keeps referring to. Node.js is *not* required, because the web service runs inside Docker; install it from nodejs.org only if you want to run the page outside a container.

CHECKPOINT 0

All four commands answer with a version, and Docker Desktop is running. Pull the database image now so it is cached before you need it — this is a few hundred megabytes and much nicer to wait for at this stage than later:

```
docker pull postgres:16-alpine
```

PART 1

Project folder and virtual environment

1 Create the folders

Open a terminal in wherever you keep projects and build the skeleton in one go.

```
mkdir loan-intake
cd loan-intake
mkdir api db web
mkdir web/app        # Windows PowerShell: use  mkdir web\app
```

Everything from here on assumes your terminal is in `loan-intake/` unless a command says otherwise. Open the folder in VS Code now (`code .`, or File → Open Folder).

2 Create and activate the virtual environment

A virtual environment gives this project its own Python and its own packages, so nothing you install here can disturb another project. Create it at the top level:

```
python -m venv .venv
```

Then activate it — the command differs by platform:

```
source .venv/bin/activate          # macOS / Linux
.venv\Scripts\activate             # Windows PowerShell or cmd
```

Your prompt should now begin with `(.venv)`. That prefix is your only reliable signal that you are talking to the project's Python rather than the machine's.

REMEMBER THIS

Activation lives in one terminal session only. Every time you open a new terminal, reopen the project, or restart your machine, run the activate command again. **Nothing is lost** — the `.venv/` folder and all its packages are still on disk; only the activation needs repeating. If imports suddenly fail with `ModuleNotFoundError`, this is almost always why.

IF IT BREAKS

On Windows PowerShell, `running scripts is disabled on this system` means the execution policy is blocking the activate script. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer yes, and activate again. On Ubuntu, `No module named venv` means you need `sudo apt install python3-venv`.

3 Declare the Python dependencies

Rather than installing loosely and freezing afterwards, write the file first — it is the honest record of what this project needs, and Docker will replay it later.

`api/requirements.txt`

```
fastapi==0.115.0           # the web framework
uvicorn[standard]==0.30.6  # the server that actually listens on a port
pydantic==2.9.2            # request/response validation
psycopg[binary]==3.2.3     # PostgreSQL driver ([binary] = no compiler needed)
pytest==8.3.3              # test runner
httpx==0.27.2              # required by FastAPI's TestClient
```

Install them into the activated environment:

```
pip install -r api/requirements.txt
```

Versions are pinned deliberately. An unpinned install works today and breaks in four months when a library releases a change — pinning is what makes “it works on my machine” reproducible on anyone else's.

4 Tell Git what to ignore, before your first commit

Do this now rather than later; it is much harder to remove a large folder from history than to never commit it.

`.gitignore` at the top level

```
# Python
.venv/
__pycache__/
*.pyc
.pytest_cache/

# Node
node_modules/
.next/

# Editor / OS
.vscode/
.DS_Store
```

Every entry is large, machine-specific and fully regenerable from the manifests you *do* commit. The rule: version the recipe, never the build output.

CHECKPOINT 1

Your prompt shows `(.venv)`, and this prints a version rather than an error:

```
python -c "import fastapi, psycopg, pytest; print(fastapi.__version__)"
```

PART 2

The FastAPI service

We build the service in two passes. First it stores applications in a Python list, so you can run it with one command and see it work immediately. In Part 4 we swap that list for PostgreSQL — the routes and models do not change. Getting something working before adding a database is a habit worth keeping.

`api/main.py` version 1 — in memory, complete file

```
"""Loan intake service — a small typed API over a list of applications.

Run it from inside the api/ folder with:
    uvicorn main:app --reload
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
```

```python
# The application object. FastAPI reads the title into the docs page.
app = FastAPI(title="Loan Intake Service", version="1.0.0")


# ── Models: these ARE the contract ──────────────────────────────
# A Pydantic model parses incoming JSON, validates every field, and
# documents itself. Field(...) attaches the rules.

class ApplicationIn(BaseModel):
    """What a client is allowed to send us."""
    applicant_name: str = Field(min_length=2, max_length=100)
    monthly_income: float = Field(gt=0)          # gt = greater than
    amount_requested: float = Field(gt=0, le=10_00_000)  # le = at most
    purpose: str = Field(min_length=2, max_length=50)


class ApplicationOut(ApplicationIn):
    """What we send back: everything above, plus server-assigned fields.

    Inheriting keeps the shared fields in one place. Separating In from
    Out is deliberate — a client must not be able to set id or status.
    """
    id: int
    status: str = "received"


# Our "database" for now: a module-level list. It is emptied whenever
# the process restarts, which is exactly why Part 4 exists.
DB: list[ApplicationOut] = []


# ── Routes ──────────────────────────────────────────────────────
# A route is a normal function plus a decorator naming the method
# and path. The return type annotation is not decoration: FastAPI
# uses it to build the response and the documentation.

@app.get("/health")
def health() -> dict[str, str]:
    """Cheap liveness check. Conventional, and used by monitoring."""
    return {"status": "ok"}


@app.post("/applications", response_model=ApplicationOut, status_code=201)
def create_application(payload: ApplicationIn) -> ApplicationOut:
    """Create one application.

    payload is typed with a Pydantic model, so FastAPI knows it comes
    from the JSON body. If the body fails validation this function is
```

```
    never called — the client gets a 422 describing what was wrong.
    201 means "created", the correct status for a successful POST.
    """
    record = ApplicationOut(id=len(DB) + 1, **payload.model_dump())
    DB.append(record)
    return record


@app.get("/applications", response_model=list[ApplicationOut])
def list_applications() -> list[ApplicationOut]:
    """Return every application.

    response_model validates and FILTERS what goes out: any field not
    declared on the model is stripped before the client sees it.
    """
    return DB


@app.get("/applications/{app_id}", response_model=ApplicationOut)
def get_application(app_id: int) -> ApplicationOut:
    """Return one application by id.

    app_id appears in the path, so FastAPI reads it from the URL and
    converts it to int. A non-numeric id gets an automatic 422.
    """
    for record in DB:
        if record.id == app_id:
            return record
    # A valid request for something that does not exist: that is a 404,
    # and it comes from our own logic rather than from validation.
    raise HTTPException(status_code=404, detail="Application not found")
```

Run it

```
cd api
uvicorn main:app --reload
```

Read the argument as “in the module `main`, use the object called `app`”. `--reload` restarts the server whenever you save a file, which is what makes editing pleasant; never use it in production. Leave this running and open a second terminal for other commands (activate the venv there too).

Try it from the browser

Open http://localhost:8000/docs. A complete interactive API explorer is already there — you wrote no documentation code. It was generated from your type hints and models.

Expand `POST /applications`, click **Try it out**, and send this:

```
{
  "applicant_name": "Asha Traders",
  "monthly_income": 45000,
  "amount_requested": 300000,
  "purpose": "working_capital"
}
```

You should get **201** back, with `id` and `status` filled in. Then `GET /applications` lists it.

Now deliberately break it

This is the most instructive minute in Part 2. Send the same body three more times, changing one thing each time:

- Set `"monthly_income": -5` → **422**, because of `gt=0`.
- Set `"amount_requested": 5000000` → **422**, because of `le=10_00_000`.
- Delete the `"purpose"` line entirely → **422**, field required.
- Request `GET /applications/99` → **404**, from your `HTTPException`.

Read one of those 422 bodies properly. It names the field, the rule that failed and a human-readable message. You wrote none of that, and it is the contract a frontend developer codes against.

IF IT BREAKS

`Address already in use` on port 8000 means something else is listening — stop it, or run `uvicorn main:app --reload --port 8001`. `Error loading ASGI app. Could not import module "main"` means you are not in the `api/` folder. `command not found: uvicorn` means the virtual environment is not activated in this terminal.

CHECKPOINT 2

From `/docs` you have seen a 201, a 422 and a 404, and you can explain where each came from. Stop the server with `Ctrl+C` when you are ready to continue.

PART 3

Tests

Two tests: one proving the happy path works, one proving bad input is refused. The second is the more valuable of the two — a happy-path test stays green even if you accidentally delete every validation rule.

api/test_main.py complete file

```
"""Tests for the loan intake service.

Run from inside api/ with:  pytest -v
"""

from fastapi.testclient import TestClient

from main import DB, app

# TestClient drives the app IN-PROCESS: no server starts, no port is
# opened, nothing touches the network. That is why this is so fast.
client = TestClient(app)


def setup_function():
    """Runs automatically before EVERY test in this file.

    Clearing shared state is what keeps tests independent. Without it,
    tests pass alone and fail together — the worst kind of test suite,
    because the result depends on the order they happen to run in.
    """
    DB.clear()


def test_create_and_list_application():
    """The happy path: a valid application is created, then listed."""
    payload = {
        "applicant_name": "Asha Traders",
        "monthly_income": 45000,
        "amount_requested": 300000,
        "purpose": "working_capital",
    }

    created = client.post("/applications", json=payload)
    assert created.status_code == 201            # created, not 200
    body = created.json()
    assert body["id"] == 1                       # server assigned it
```

```
    assert body["status"] == "received"          # default applied
    assert body["applicant_name"] == "Asha Traders"

    listed = client.get("/applications")
    assert listed.status_code == 200
    assert len(listed.json()) == 1


def test_negative_income_is_rejected():
    """The contract: invalid input never becomes a stored application."""
    bad = {
        "applicant_name": "X Ltd",
        "monthly_income": -1,                    # violates gt=0
        "amount_requested": 1000,
        "purpose": "vehicle",
    }
    response = client.post("/applications", json=bad)
    assert response.status_code == 422

    # And nothing was stored as a side effect.
    assert client.get("/applications").json() == []


def test_missing_application_returns_404():
    """A valid request for something that does not exist."""
    assert client.get("/applications/999").status_code == 404
```

Run them

```
cd api          # if you are not already there
pytest -v
```

Expect three passes in well under a second:

```
test_main.py::test_create_and_list_application PASSED      [ 33%]
test_main.py::test_negative_income_is_rejected PASSED      [ 66%]
test_main.py::test_missing_application_returns_404 PASSED  [100%]

======================  3 passed in 0.34s  ======================
```

Note that you did not start the server. pytest found the files itself: the convention is files named `test_*.py` containing functions named `test_*`, using plain `assert`. That is the entire API you need today.

PROVE THE TESTS ARE REAL

A test you have never seen fail is not yet trustworthy. Open `main.py`, change `monthly_income: float = Field(gt=0)` to plain `monthly_income: float`, and run `pytest` again — the rejection test should now fail, because you just deleted the rule it guards. Put the constraint back and watch it go green. That red-then-green cycle is the whole point of having tests.

IF IT BREAKS

`ModuleNotFoundError: No module named 'main'` means you are running pytest from the project root rather than from `api/`. `ImportError ... httpx` means the `httpx` dependency is missing — `TestClient` is built on it; re-run the pip install from Part 1.

CHECKPOINT 3

Three tests pass, and you have watched one fail on purpose and recovered it.

PART 4

A real database

The list in `main.py` disappears every time the process restarts. Now we replace it with PostgreSQL, running in a container so you do not have to install a database on your machine.

1 The schema

Postgres runs any `.sql` file it finds in a special folder the very first time it initialises an empty database. We mount our file there in Part 6.

`db/init.sql` complete file

```
-- Runs ONCE, on first initialisation of an empty database.
-- This is not a migration system: editing it later will not change
-- an existing database. Real projects use migration tools instead.

CREATE TABLE IF NOT EXISTS applications (
    -- SERIAL PRIMARY KEY: Postgres assigns a unique, auto-incrementing
    -- id. The client never invents one.
    id               SERIAL PRIMARY KEY,
    applicant_name   TEXT    NOT NULL,

    -- The CHECK is the same rule as Pydantic's gt=0, one layer deeper.
    -- Defence in depth: Pydantic gives the user a friendly error, this
    -- holds even for data that never came through our API.
    monthly_income   NUMERIC NOT NULL CHECK (monthly_income > 0),
    amount_requested NUMERIC NOT NULL CHECK (amount_requested > 0),
    purpose          TEXT    NOT NULL,
    status           TEXT    NOT NULL DEFAULT 'received'
);

-- Two seed rows, so the page has something to show immediately.
INSERT INTO applications
    (applicant_name, monthly_income, amount_requested, purpose)
VALUES
    ('Asha Traders',    45000, 300000, 'working_capital'),
    ('Ravi Auto Works', 62000, 450000, 'equipment');
```

2 The connection helper

api/db.py complete file

```
"""Database connection helper.

Kept separate from main.py so the routes stay readable.
"""

import os

import psycopg
from psycopg.rows import dict_row

# Read the connection string from the environment, with a sensible
# local default. Docker Compose sets this variable in Part 6 — which
# is how the same code works both on your machine and in a container.
#
# Format: postgresql://user:password@host:port/database
DATABASE_URL = os.environ.get(
    "DATABASE_URL",
    "postgresql://intake:intake@localhost:5432/intake",
)


def get_conn() -> psycopg.Connection:
    """Open a new connection.

    row_factory=dict_row makes each result row a dict rather than a
    plain tuple, so we can write ApplicationOut(**row) instead of
    unpacking by position.

    One connection per request is fine at this scale. A production
    service would use a connection pool.
    """
    return psycopg.connect(DATABASE_URL, row_factory=dict_row)
```

3 Rewrite main.py against the database

Replace `api/main.py` with the version below. Compare it to version 1 as you go: the models are identical and the routes have the same names, paths and status codes. Only the bodies changed — and CORS was added, which Part 5 explains.

`api/main.py` version 2 — PostgreSQL, complete file, replaces version 1

```
"""Loan intake service — FastAPI over PostgreSQL."""

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field
```

```
from db import get_conn

app = FastAPI(title="Loan Intake Service", version="2.0.0")

# ── CORS ──────────────────────────────────────────────────────────
# Our page is served from port 3000 and this API listens on 8000.
# Because the ports differ, browsers treat them as different ORIGINS
# and block the page from reading our responses until we opt in.
# The browser enforces this, not FastAPI — the same request from curl
# works fine either way.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],  # never "*" in production
    allow_methods=["*"],
    allow_headers=["*"],
)


# ── Models: unchanged from version 1 ──────────────────────────────

class ApplicationIn(BaseModel):
    applicant_name: str = Field(min_length=2, max_length=100)
    monthly_income: float = Field(gt=0)
    amount_requested: float = Field(gt=0, le=10_00_000)
    purpose: str = Field(min_length=2, max_length=50)


class ApplicationOut(ApplicationIn):
    id: int
    status: str = "received"


# ── Routes ────────────────────────────────────────────────────────

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/applications", response_model=list[ApplicationOut])
def list_applications() -> list[ApplicationOut]:
    """Read every application, oldest first."""
    # 'with' guarantees the connection closes even if an error is raised.
    with get_conn() as conn:
        rows = conn.execute(
            "SELECT * FROM applications ORDER BY id"
        ).fetchall()
```

```
    # Each row is a dict, so ** unpacks it into keyword arguments.
    return [ApplicationOut(**row) for row in rows]


@app.post("/applications", response_model=ApplicationOut, status_code=201)
def create_application(payload: ApplicationIn) -> ApplicationOut:
    """Insert one application and return it, id included."""
    with get_conn() as conn:
        row = conn.execute(
            # %(name)s placeholders — NEVER an f-string. The driver
            # sends the query shape and the values separately, so a
            # value can never be read as SQL. This is what prevents
            # SQL injection.
            """
            INSERT INTO applications
                (applicant_name, monthly_income,
                 amount_requested, purpose)
            VALUES
                (%(applicant_name)s, %(monthly_income)s,
                 %(amount_requested)s, %(purpose)s)
            RETURNING *
            """,
            payload.model_dump(),   # the model as a plain dict
        ).fetchone()
        conn.commit()               # make the change permanent
    return ApplicationOut(**row)


@app.get("/applications/{app_id}", response_model=ApplicationOut)
def get_application(app_id: int) -> ApplicationOut:
    """Read one application by id, or 404."""
    with get_conn() as conn:
        row = conn.execute(
            "SELECT * FROM applications WHERE id = %(id)s",
            {"id": app_id},
        ).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="Application not found")
    return ApplicationOut(**row)
```

NOTE ON THE TESTS

Your Part 3 tests import `DB`, which no longer exists — they will now fail to import. That is expected and correct. Testing against a real database needs a fixture that resets the table between tests, which is a stretch goal rather than part of the core path; for now, comment out the `DB` import and the `setup_function`, and keep the 404 test.

4 Start just the database and try it

You can run Postgres alone, before writing any Compose file, with a single command. From the project root:

```
docker run --name intake-db -e POSTGRES_USER=intake \
  -e POSTGRES_PASSWORD=intake -e POSTGRES_DB=intake \
  -v "$(pwd)/db/init.sql:/docker-entrypoint-initdb.d/init.sql" \
  -p 5432:5432 -d postgres:16-alpine
```

On Windows PowerShell, replace the line continuations with one long line and `$(pwd)` with `${PWD}`. Then run the API against it:

```
cd api
uvicorn main:app --reload
```

Visit localhost:8000/applications and you should see the two seed rows. Submit a new one from `/docs`, then **restart the server** and list again — it is still there. That persistence is the whole point of this part.

When you are done, stop and remove the container: `docker rm -f intake-db`. Compose will manage it from here.

IF IT BREAKS

`connection refused` means Postgres is not up yet (give it a few seconds) or the container is not running (`docker ps` to check). `password authentication failed` means the credentials in `DATABASE_URL` do not match the ones you passed to `docker run`. `relation "applications" does not exist` means the mount path was wrong, so `init.sql` never ran — remove the container with `docker rm -f intake-db`, fix the `-v` path, and start it again. Note that the seed only runs on a *fresh* database.

CHECKPOINT 4

A submitted application survives a restart of the API server. Your data lives in Postgres now, not in Python.

PART 5

The web page

Four small config files and two React components. You do not need Node installed — the container builds it in Part 6 — but the files must exist first.

web/package.json

```
{
  "name": "loan-intake-web",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "14.2.5",
    "react": "18.3.1",
    "react-dom": "18.3.1"
  },
  "devDependencies": {
    "typescript": "5.5.4",
    "@types/react": "18.3.3",
    "@types/node": "20.14.10"
  }
}
```

web/next.config.js

```
/** @type {import('next').NextConfig} */
module.exports = {};   // defaults are fine for this project
```

web/tsconfig.json

```
{
  "compilerOptions": {
    "target": "es2017",
    "lib": ["dom", "es2017"],
    "jsx": "preserve",
    "module": "esnext",
    "moduleResolution": "bundler",
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "incremental": true,
    "plugins": [{ "name": "next" }]
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx"],
  "exclude": ["node_modules"]
}
```

web/app/layout.tsx the shell every page is wrapped in

```
import type { ReactNode } from "react";

export const metadata = { title: "Loan Intake" };

export default function RootLayout(props: { children: ReactNode }) {
  return (
    <html lang="en">
      <body style={ { fontFamily: "system-ui, sans-serif", margin: 32 } }>
        {props.children}
      </body>
    </html>
  );
}
```

Note the doubled braces in `style={ {...} }` — the outer pair is JSX saying "a JavaScript expression follows", the inner pair is the object literal itself. Written without the space, as you normally would, it is `style=`.

web/app/page.tsx complete file

```
"use client";
// "use client" marks this a client component: it runs in the browser,
// which is what lets it hold state and respond to typing.

import { useEffect, useState } from "react";
```

```
// The shape the API returns. Keeping this in sync with the Pydantic
// model is what "the contract" means in practice.
type Application = {
  id: number;
  applicant_name: string;
  monthly_income: number;
  amount_requested: number;
  purpose: string;
  status: string;
};

// Next.js only exposes env vars to browser code when the name starts
// with NEXT_PUBLIC_. Docker Compose sets this in Part 6; the fallback
// covers running the page outside a container.
const API = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";

export default function Home() {
  const [apps, setApps] = useState<Application[]>([]);
  const [name, setName] = useState("");
  const [error, setError] = useState("");

  // Fetch the list from the API and put it in state.
  async function load() {
    try {
      const response = await fetch(`${API}/applications`);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      setApps(await response.json());
      setError("");
    } catch (e) {
      // A failure here is usually the API being down, or CORS.
      setError("Could not reach the API. Is it running?");
    }
  }

  // Run load() once, after the first render. The empty array is the
  // dependency list: nothing to watch, so it never repeats.
  useEffect(() => {
    load();
  }, []);

  async function submit() {
    if (name.trim().length < 2) {
      setError("Name must be at least 2 characters.");
      return;
    }
    const response = await fetch(`${API}/applications`, {
      method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        applicant_name: name,
        monthly_income: 50000,
        amount_requested: 200000,
        purpose: "working_capital",
      }),
    });
    if (response.status === 422) {
      // The API rejected it. A real UI would read response.json()
      // and show the specific field errors it names.
      setError("The API rejected that input (422).");
      return;
    }
    setName("");     // clear the box
    load();          // and re-read the list
  }

  return (
    <main>
      <h1>Loan applications</h1>

      <div style={ { display: "flex", gap: 8, margin: "16px 0" } }>
        <input
          value={name}
          placeholder="Applicant name"
          sc-camel-on-change={(e) => setName(e.target.value)}
        />
        <button sc-camel-on-click={submit}>Submit</button>
      </div>

      {error && <p style={ { color: "crimson" } }>{error}</p>}

      <ul>
        {apps.map((a) => (
          // key lets React track list items efficiently.
          <li key={a.id}>
            #{a.id} — {a.applicant_name} ({a.status})
          </li>
        ))}
      </ul>

      {apps.length === 0 && !error && <p>No applications yet.</p>}
    </main>
  );
}
```

CHECKPOINT 5

All six `web/` files exist. Nothing to run yet — Part 6 builds and starts this.

PART 6

One command starts everything

Two Dockerfiles describe how to build the api and web images; one Compose file wires all three services together on a shared private network.

`api/Dockerfile`

```
# The base image: a slim Python 3.11.
FROM python:3.11-slim

WORKDIR /app

# Copy the dependency list FIRST and install it, before copying the
# source. Docker caches each step, so editing your Python code no
# longer re-installs every package — rebuilds take seconds instead
# of minutes. Reverse these two blocks and you lose that entirely.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Now the source.
COPY . .

# --host 0.0.0.0 is REQUIRED. By default uvicorn listens only on
# 127.0.0.1, which inside a container means "this container only" —
# nothing outside could reach it, including your browser.
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

`web/Dockerfile`

```
FROM node:20-alpine

WORKDIR /app

# Same caching trick: manifests first, then install, then the source.
COPY package*.json ./
RUN npm install

COPY . .

# Dev server, because this is a teaching project. A production build
# would run `npm run build` and then `npm start`.
CMD ["npm", "run", "dev"]
```

docker-compose.yml at the top level — complete file

```
# Three services on one private network. Compose gives each a DNS
# name equal to its service name, which is how they find each other.

services:

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: intake
      POSTGRES_PASSWORD: intake
      POSTGRES_DB: intake
    volumes:
      # Mount our schema into the folder Postgres runs on first boot.
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
      # Published so you can connect from your machine if you want to.
      - "5432:5432"
    healthcheck:
      # "Started" is not "ready": Postgres takes a moment to accept
      # connections. This command is retried until it succeeds.
      test: ["CMD-SHELL", "pg_isready -U intake"]
      interval: 2s
      timeout: 3s
      retries: 15

  api:
    build: ./api
    environment:
      # NOTE THE HOSTNAME: 'db', not 'localhost'. Inside this network
      # the database is reachable by its service name. 'localhost'
```

```
      # here would mean the api's own container, where nothing listens.
      DATABASE_URL: postgresql://intake:intake@db:5432/intake
    ports:
      - "8000:8000"
    depends_on:
      db:
        # Wait for the healthcheck to pass, not merely for the
        # container to start. Without this the api can race Postgres
        # and crash on its first query.
        condition: service_healthy

  web:
    build: ./web
    environment:
      # NOTE THE HOSTNAME: 'localhost', because this value is used by
      # the BROWSER, which runs on your machine — outside this network.
      NEXT_PUBLIC_API_URL: http://localhost:8000
    ports:
      - "3000:3000"
    depends_on:
      - api
```

THE ONE THING TO UNDERSTAND HERE

Two different hostnames appear for two different journeys. The **api** reaches the database at `db` because both are inside the Compose network. The **browser** reaches the api at `localhost:8000` because it is outside that network and uses the published port. Get these backwards and you get `connection refused` — it is the most common failure in this whole build.

Bring it all up

From the project root, with Docker Desktop running:

```
docker compose up --build
```

The first run takes several minutes — it builds two images and installs all dependencies. Later runs take seconds. You will see interleaved logs from all three services, colour-coded by name; `db` announcing it is ready to accept connections, then the api starting, then Next.js compiling.

Then open http://localhost:3000. You should see the two seed applications. Type a name, click Submit, and it appears in the list. **Refresh the page** — it is still there, because it went to Postgres.

```
docker compose up          # start again (no rebuild)
docker compose up -d       # start in the background
docker compose logs -f api # follow one service's logs
docker compose ps          # what is running, and is it healthy
docker compose down        # stop and remove the containers
docker compose down -v     # ...and DELETE the database volume
```

CHECKPOINT 6

One command brought up three services; you submitted an application in the browser and it survived a page refresh. The loop is closed.

PART 7

README and Git

README.md

```
# Loan Intake

A typed FastAPI service over PostgreSQL, with a Next.js page that
calls it. All three run together under Docker Compose.

## Run it

    docker compose up --build

- Web page: http://localhost:3000
- API docs: http://localhost:8000/docs

## Tests

    cd api
    pip install -r requirements.txt
    pytest -v

## Layout

- `api/`  FastAPI service (`main.py`, `db.py`, `test_main.py`)
- `db/`   `init.sql` — schema and seed rows, run on first boot
- `web/`  Next.js page (`app/page.tsx`)
```

A README that gets someone from clone to running in one command is the difference between a project that can be handed over and one that cannot. Write it for a stranger.

Commit and push

```
git init
git add -A
git commit -m "Loan intake: FastAPI + Postgres + Next.js under Compose"
```

Before pushing, check what you actually staged — this catches a missing `.gitignore` immediately:

```
git ls-files | head -30
```

You should see your sixteen files and **nothing** from `.venv/`, `node_modules/` or `__pycache__/`. If you do see them, add the entry to `.gitignore`, then `git rm -r --cached .venv node_modules` and commit again.

Now create an empty repository on GitHub — no README, no `.gitignore`, since you already have both — and push:

```
git remote add origin https://github.com/<you>/loan-intake.git
git branch -M main
git push -u origin main
```

THE THREE COMMANDS ARE THREE DIFFERENT THINGS

`add` stages what you intend to record. `commit` writes that snapshot into your *local* history. `push` sends local commits to the server. Work that is committed but never pushed still exists only on your laptop — which is not a backup.

CHECKPOINT 7 · THE REAL ONE

Clone your own repository into a *different* folder, run `docker compose up --build`, and use it at `localhost:3000`. If that works, your project is genuinely reproducible — which is the actual standard, and more than many working codebases manage.

REFERENCE

Troubleshooting

Symptoms you are most likely to hit, and what they actually mean.

SYMPTOM	CAUSE AND FIX
`ModuleNotFoundError`	Virtual environment not activated in this terminal. Re-run the activate command — nothing is lost.
`command not found: uvicorn`	Same cause as above, or the pip install has not been run.
`Could not import module "main"`	You are not in the `api/` folder when running uvicorn or pytest.
`Address already in use`	Another program holds that port. Find it with `docker ps`, or change the host side of the port mapping.
`Cannot connect to the Docker daemon`	Docker Desktop is not running. Open it and wait for green.
`connection refused` from the api	Almost always `localhost` where it should be `db` in `DATABASE_URL`. Otherwise a missing healthcheck condition.
`relation "applications" does not exist`	`init.sql` never ran — check the volume path. It only runs on a fresh database: `docker compose down -v`, then up again.
Page loads, list is empty, console shows CORS	`allow_origins` does not match the page's origin exactly — scheme, host and port all count.
Page cannot reach the API at all	`NEXT_PUBLIC_API_URL` must be `http://localhost:8000`, not `http://api:8000` — the browser is outside the network.
Everything 422s unexpectedly	Read the response body — it names the field and the rule. Usually a typo in a JSON key.
Rebuild is slow every time	The dependency `COPY` is below the source `COPY` in your Dockerfile, so the cache is invalidated on every edit.

WHERE TO GO NEXT

Extend it

Once the build works end to end, these are the natural next steps — in rough order of usefulness.

1. **Test against the database.** Add a pytest fixture that truncates `applications` before each test, and restore the tests you commented out in Part 4.
2. **Add a typed filter.** `GET /applications?purpose=equipment` — a query parameter with a default. Validation and documentation come for free.
3. **Use the Enum.** Change `purpose: str` to a `LoanPurpose(str, Enum)` and watch `/docs` turn it into a dropdown of valid values.
4. **Show real validation errors.** Read the 422 body in `page.tsx` and display the specific field messages the API returns.
5. **Add a named volume** to the `db` service so data survives `docker compose down`.
6. **Add pagination** — `?limit=` and `?offset=` — and an index on the column you sort by.
7. **Run the tests in CI** on every push, so a broken contract is caught before anyone pulls it.

If you get stuck on any part of this, note down exactly which checkpoint failed and what the error said — that is a far better question than “it does not work”, and you will usually find the answer yourself while writing it down.